

HERANÇA E POLIMORFISMO

É muito comum durante o desenvolvimento de software, identificarmos classes e objetos que possuem características comuns. Quando nos deparamos com este tipo de situação podemos fazer uso do conceito de herança e polimorfismo para criar uma hierarquia de classes e desenvolver métodos com um comportamento específico para cada tipo de classe criada. A seguir vamos verificar como fazer uso da Herança e do Polimorfismo e praticar alguns exemplos.

2.1 HERANÇA

A herança é o conceito da orientação a objetos que permite que classes herdem ou assumam as características (atributos e métodos) de uma outra classe. Esta classe que "empresta" suas características às demais classes é conhecida como **superclasse** e as classes que herdam suas características são chamadas de **subclasses**.

Com a herança, a superclasse possuirá uma estrutura com atributos e métodos e esta estrutura será absorvida pela subclasse. A subclasse será sempre uma representação da superclasse, porém poderá também ter outros métodos e atributos específicos.

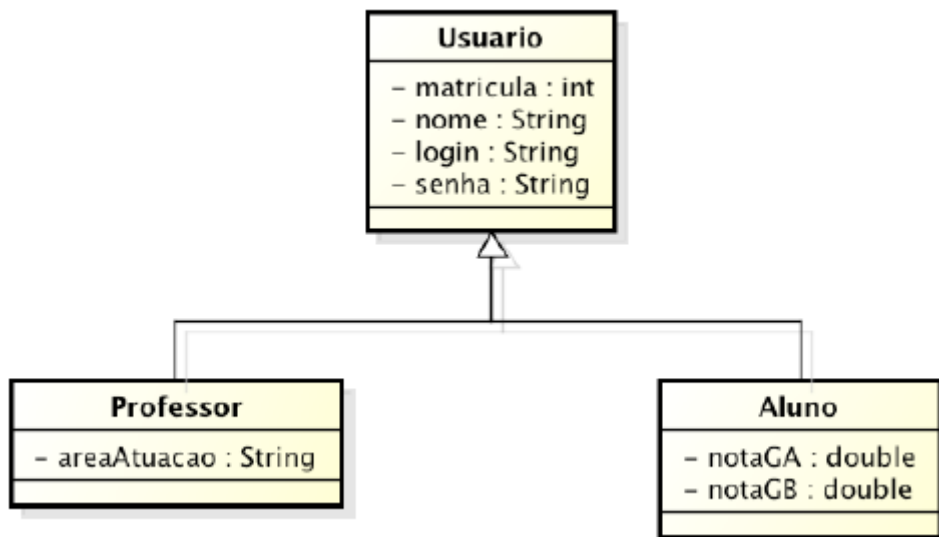
Este relacionamento de herança entre superclasse e subclasses é chamado de generalização, onde a superclasse é a classe genérica e a subclasse é a classe especialista.

Para exemplificarmos o uso da herança, vamos utilizar o cenário de um sistema como o SGA, por exemplo, Este sistema é acessado tanto por Professores quanto por Alunos. Ambos, Professores e Alunos são Usuários do sistema SGA. E ambos, possuem algumas características semelhantes que são padrões para um usuário do sistema. Por exemplo, todos os usuários do sistema possuem matrícula e nome. O Professor possui a matrícula referente ao seu contrato com a instituição e o aluno possui a matrícula referente a sua matrícula como aluno na instituição. Além disso, ambos utilizam um login e uma senha para acesso ao sistema. Esta característica também é comum para os usuários do sistema. Vamos então verificar quais seriam as características de Professores e Alunos que são comuns para os usuários do sistema:

Professor	Aluno
Matrícula = número do seu contrato)	Matrícula = número de sua matrícula como aluno)
Nome = nome do Professor	Nome = nome do Aluno
Login = login do Professor utilizado para acesso ao sistema	Login = login do Aluno utilizado para acesso ao sistema
Senha = senha do Professor utilizado para acesso ao sistema	Senha = senha do Aluno utilizado para acesso ao sistema

Conforme representado na tabela, as características: matrícula, nome, login e senha são comuns para Professor e Aluno. Então, podemos afirmar que todo usuário do sistema, independente se for Professor ou Aluno possuem as características: matrícula, nome, login e senha. Desta forma, podemos criar uma superclasse chamada: Usuario com estas características, e duas subclasses uma para o Professor e outra para o Aluno.

A figura abaixo representa como ficaria esta herança entre a superclasse Usuario e as subclasses: Professor e Aluno:



Na representação acima, as classes: **Professor e Aluno são classes especialistas**, ou seja, estas classes possuem os quatro atributos da classe Usuario além dos atributos específicos de cada classe. Perceba que na classe: Usuario foi mantido somente as características comuns entre todos os usuários e, nas classes: Professor e Aluno foi incluído somente as características específicas destes tipos de usuários.

Para representar a herança no código Java utilizamos a palavra reservada: **extends** na assinatura da subclasse, conforme a sintaxe abaixo:

```
//a declaração superclasse não muda
public class Usuario {
    ...
}

//na declaração da subclasse acrescentamos a palavra: extends após o nome da subclasse
public class Professor extends Usuario{
    ...
}

//na declaração da subclasse acrescentamos a palavra: extends após o nome da subclasse
public class Aluno extends Usuario{
    ...
}
```

2.2 PRIMEIRO EXEMPLO UTILIZANDO HERANÇA

Vamos iniciar a prática verificando como utilizar a HERANÇA em Java. Para tanto, abra o BlueJ e crie um novo projeto com o nome: PraticaHeranca. Vamos utilizar a classe **Teclado**, para solicitar os dados do teclado. Portanto, resgate essa classe nas aulas anteriores.

Crie a classe **Usuario** com os quatro atributos e os métodos de configuração e acesso aos atributos, conforme figura abaixo:

```
public class Usuario
{
    // atributos comuns para todos os usuários
    private int matricula;
    private String nome;
    private String login;
    private String senha;

    //método construtor para todos os usuários
    public Usuario(int mat, String nom, String log, String sen)
    {
        matricula = mat;
        nome = nom;
        login = log;
        senha = sen;
    }
}
```

```

//método de acesso ao atributo: matrícula
public int getMatricula()
{
    return matricula;
}

//método de configuração do atributo: matrícula
public void setMatricula(int mat)
{
    matricula = mat;
}

//método de acesso ao atributo: nome
public String getNome()
{
    return nome;
}

//método de configuração do atributo: nome
public void setNome(String nom)
{
    nome = nom;
}

//método de acesso ao atributo: login
public String getLogin()
{
    return login;
}

//método de configuração do atributo: login
public void setLogin(String log)
{
    login = log;
}

//método de acesso ao atributo: senha
public String getSenha()
{
    return senha;
}

//método de configuração do atributo: senha
public void setSenha(String sen)
{
    senha = sen;
}
}

```

Compile a classe: Usuario e crie a classe: **Professor**, conforme a figura abaixo:

```

public class Professor extends Usuario
{
    // atributos específicos para o Professor
    private String areaAtuacao;

    //método construtor Professor = recebe todos os parâmetros do construtor da superclasse = construtor da classe: Usuario
    public Professor(int mat, String nom, String log, String sen)
    {
        //encaminha os parâmetros para o construtor da superclasse = construtor da classe: Usuario
        super(mat, nom, log, sen);
    }

    //método de acesso ao atributo: areaAtuacao
    public String getAreaAtuacao()
    {
        return areaAtuacao;
    }

    //método de configuração do atributo: areaAtuacao
    public void setAreaAtuacao(String are)
    {
        areaAtuacao = are;
    }
}

```

Compile a classe: Professor e crie a classe: **Aluno**, conforme a figura abaixo:

```

public class Aluno extends Usuario
{
    // atributos específicos do aluno
    private double notaGA;
    private double notaGB;

    //método construtor Aluno = recebe todos os parâmetros do construtor da superclasse = construtor da classe: Usuario
    public Aluno(int mat, String nom, String log, String sen)
    {
        //encaminha os parâmetros para o construtor da superclasse = construtor da classe: Usuario
        super(mat, nom, log, sen);
    }

    //método de acesso ao atributo: notaGA
    public double getNotaGA()
    {
        return notaGA;
    }

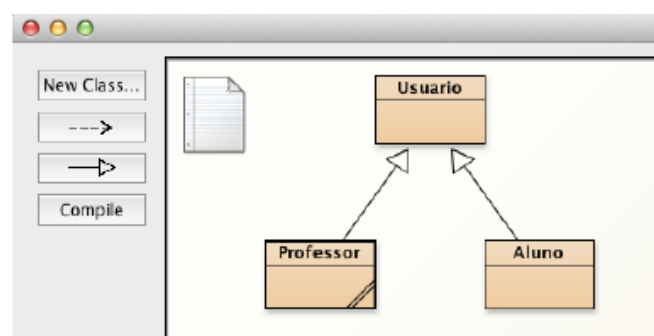
    //método de configuração do atributo: notaGA
    public void setNotaGA(double not)
    {
        notaGA = not;
    }

    //método de acesso ao atributo: notaGB
    public double getNotaGB()
    {
        return notaGB;
    }

    //método de configuração do atributo: notaGB
    public void setNotaGB(double not)
    {
        notaGB = not;
    }
}

```

Compile e feche a classe Aluno e observe a tela do BlueJ conforme a figura abaixo.



Veja que o BlueJ representa a estrutura de Herança, com base no diagrama de classes. Neste diagrama é possível identificar qual é a superclasse, ou classe genérica com as características comuns para todas as classes; e as subclasses. Por fim, vamos agora criar uma classe de teste para criarmos os objetos: Professores e Alunos no nosso sistema. Crie a classe de teste: **PrimeiroExemploHeranca**, conforme figura abaixo.

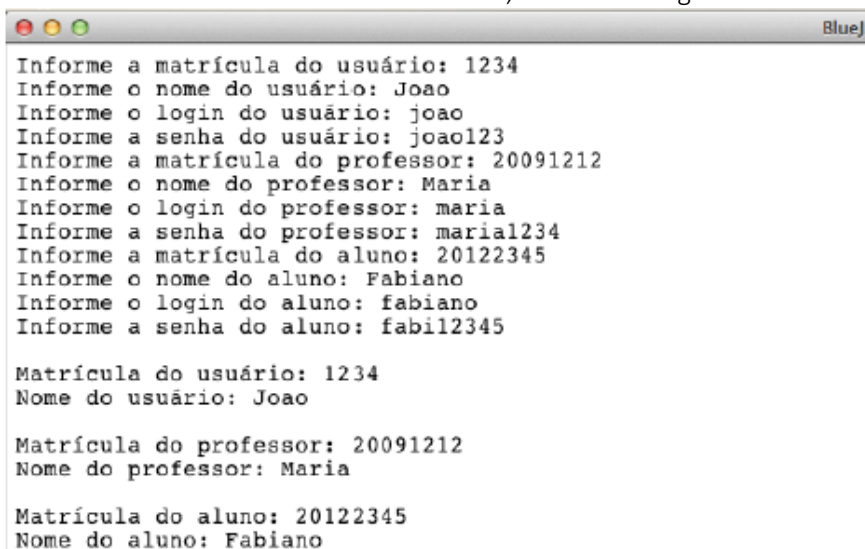
```
public class PrimeiroExemploHeranca
{
    public static void main(String args[])
    {
        //criar um novo usuário no sistema
        Usuario usuario = new Usuario( Teclado.leInt("Informe a matrícula do usuário: "),
                                         Teclado.leString("Informe o nome do usuário: "),
                                         Teclado.leString("Informe o login do usuário: "),
                                         Teclado.leString("Informe a senha do usuário: ")
                                         );

        //criar um novo usuário do tipo Professor no sistema
        Professor professor = new Professor( Teclado.leInt("Informe a matrícula do professor: "),
                                              Teclado.leString("Informe o nome do professor: "),
                                              Teclado.leString("Informe o login do professor: "),
                                              Teclado.leString("Informe a senha do professor: ")
                                              );

        //criar um novo usuário do tipo Aluno no sistema
        Aluno aluno = new Aluno( Teclado.leInt("Informe a matrícula do aluno: "),
                                  Teclado.leString("Informe o nome do aluno: "),
                                  Teclado.leString("Informe o login do aluno: "),
                                  Teclado.leString("Informe a senha do aluno: ")
                                  );

        //apresentar a matrícula e o nome da cada usuário criado
        //Usuario
        System.out.println("");
        System.out.println("Matrícula do usuário: " + usuario.getMatricula());
        System.out.println("Nome do usuário: " + usuario.getNome());
        //Professor
        System.out.println("");
        System.out.println("Matrícula do professor: " + professor.getMatricula());
        System.out.println("Nome do professor: " + professor.getNome());
        //Aluno
        System.out.println("");
        System.out.println("Matrícula do aluno: " + aluno.getMatricula());
        System.out.println("Nome do aluno: " + aluno.getNome());
    }
}
```

A partir de agora, podemos compilar a classe e executar o método main para ver o resultado da execução. Ao executar pela primeira vez, o sistema irá solicitar os dados de cada usuário, conforme a figura abaixo:



```
BlueJ:
Informe a matrícula do usuário: 1234
Informe o nome do usuário: Joao
Informe o login do usuário: joao
Informe a senha do usuário: joao123
Informe a matrícula do professor: 20091212
Informe o nome do professor: Maria
Informe o login do professor: maria
Informe a senha do professor: maria1234
Informe a matrícula do aluno: 20122345
Informe o nome do aluno: Fabiano
Informe o login do aluno: fabiano
Informe a senha do aluno: fabi12345

Matrícula do usuário: 1234
Nome do usuário: Joao

Matrícula do professor: 20091212
Nome do professor: Maria

Matrícula do aluno: 20122345
Nome do aluno: Fabiano
```

Após informarmos os dados, será apresentado na tela: a matrícula e o nome de cada usuário por meio dos métodos de acesso aos atributos. Verifique porém que tanto a classe Professor quanto a classe Aluno não possuem os métodos de acesso aos atributos: matrícula e nome. Estes métodos estão disponíveis apenas na classe: Usuario. Porém, como estas classes são subclasses da classe Usuario, elas herdam todas as características da classe Usuario e, portanto, podem fazer uso de todos os métodos que estão disponíveis nesta classe.

2.3 POLIMORFISMO

Como vimos no tópico anterior, por meio da Herança as subclasses poderão fazer uso dos métodos que estão disponíveis na superclasse. Comprovamos esta situação com base no exemplo das classes: Usuario, Professor e Aluno. No entanto, se precisarmos que um determinado método, já presente na superclasse, tenha um comportamento um pouco diferente nas subclasses, poderemos fazer uso do conceito do polimorfismo para desenvolver este comportamento específico para as subclasses.

Este tipo de situação também é conhecido como: Sobrescrita de Métodos. **Atenção:** a sobrescrita de métodos acontece quando um método existe na superclasse e é reescrito na subclasse para alterar o comportamento conforme o tipo de objeto. A assinatura do método deve ser **SEMPRE** igual tanto na superclasse quanto na subclasse, sendo apenas o seu conteúdo – instruções dentro do bloco de comandos do método – diferente. Vamos então desenvolver o segundo exemplo para praticarmos o conceito do polimorfismo e sobrescrita de métodos.

2.4 SEGUNDO EXEMPLO UTILIZANDO POLIMORFISMO

Vamos então criar um novo método na classe Usuario para praticar o uso do: **polimorfismo**. Vamos criar o método: `exibeDados()` para que seja possível apresentar os dados do usuário. Para isso, abra a classe: Usuario e crie o método: `exibeDados()`, conforme figura abaixo.

```
public void exibeDados()
{
    System.out.println("");
    System.out.println("Matrícula: " + matricula);
    System.out.println("Nome: " + nome);
    System.out.println("Login: " + login);
}
```

Compile a classe: Usuario. Agora vamos criar uma classe de teste chamada: **SegundoExemploPolimorfismo** para testar o uso deste método. A classe de teste será desenvolvida conforme a figura abaixo.

```
public class SegundoExemploPolimorfismo
{
    public static void main(String args[])
    {
        //criar um novo usuário no sistema
        Usuario usuario = new Usuario( Teclado.leInt("Informe a matrícula do usuário: "),
                                       Teclado.leString("Informe o nome do usuário: "),
                                       Teclado.leString("Informe o login do usuário: "),
                                       Teclado.leString("Informe a senha do usuário: ")
                                       );

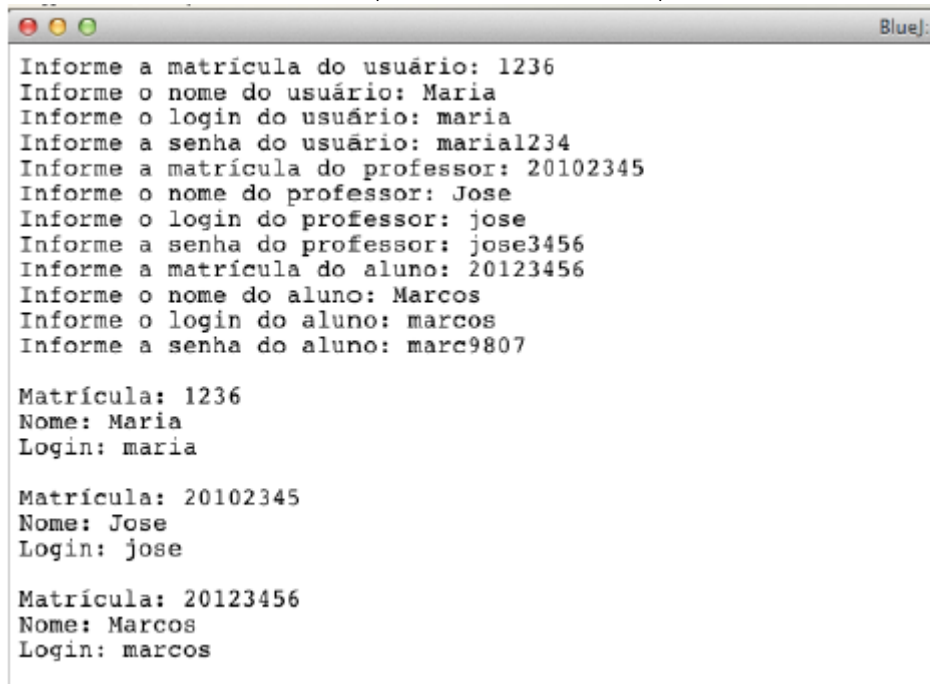
        //criar um novo usuário do tipo Professor no sistema
        Professor professor = new Professor( Teclado.leInt("Informe a matrícula do professor: "),
                                             Teclado.leString("Informe o nome do professor: "),
                                             Teclado.leString("Informe o login do professor: "),
                                             Teclado.leString("Informe a senha do professor: ")
                                             );

        //criar um novo usuário do tipo Aluno no sistema
        Aluno aluno = new Aluno( Teclado.leInt("Informe a matrícula do aluno: "),
                                Teclado.leString("Informe o nome do aluno: "),
                                Teclado.leString("Informe o login do aluno: "),
                                Teclado.leString("Informe a senha do aluno: ")
                                );

        //apresentar os dados de cada usuário criado
        //Usuario
        usuario.exibeDados();
        //Professor
        professor.exibeDados();
        //Aluno
        aluno.exibeDados();
    }
}
```

Verifique que utilizamos o mesmo cenário da primeira classe de teste de exemplo, apenas modificando as últimas instruções. Ao invés de utilizarmos os métodos de acesso aos atributos, estaremos utilizando o método `exibeDados()` criado na classe `Usuario`.

Vamos então compilar e executar a classe de teste para verificar o seu comportamento.



```
Informe a matrícula do usuário: 1236
Informe o nome do usuário: Maria
Informe o login do usuário: maria
Informe a senha do usuário: marial234
Informe a matrícula do professor: 20102345
Informe o nome do professor: Jose
Informe o login do professor: jose
Informe a senha do professor: jose3456
Informe a matrícula do aluno: 20123456
Informe o nome do aluno: Marcos
Informe o login do aluno: marcos
Informe a senha do aluno: marc9807

Matrícula: 1236
Nome: Maria
Login: maria

Matrícula: 20102345
Nome: Jose
Login: jose

Matrícula: 20123456
Nome: Marcos
Login: marcos
```

Ao executarmos o método `main`, serão solicitados os dados de cada usuário. Depois de informarmos os dados de cada usuário, será exibido os dados destes usuários, com base nas instruções que foram programadas no método `exibeDados()` da superclasse: `Usuario`. Muito bem, e se quisermos que o método `exibeDados()` tenha um comportamento diferente para o `Professor`? Neste caso, teremos que fazer uso do polimorfismo e sobrescrita de métodos para permitir este tipo de situação. Para isso, teremos que criar o método `exibeDados()` com a mesma assinatura em cada subclasse, porém no seu conteúdo – bloco de comandos – incluiremos as instruções que possibilitarão o comportamento diferente – ou específico – para cada tipo de objeto. Vamos então criar o método `exibeDados()` para a subclasse: `Professor`. O método será desenvolvido conforme a figura abaixo.

```
//método sobrescrito = deve possuir a mesma assinatura do método da superclasse: Usuario
public void exibeDados()
{
    System.out.println("");
    System.out.println("+++++ Dados do Professor +++++");

    //vamos reaproveitar as instruções que já existem no método da superclasse, para isso, vamos fazer uso da palavra: super
    super.exibeDados(); // esta instrução chama o método exibeDados() da superclasse

    //se a área de atuação for diferente de null e vazia, vamos apresentá-la também
    if(areaAtuacao!=null && !areaAtuacao.isEmpty())
        System.out.println("Área de Atuação: " + areaAtuacao);
}
```

Vamos então compilar a classe `Professor` e executar novamente o método `main` da nossa segunda classe de teste. O comportamento desta execução é apresentado na figura abaixo.

```
Informe a matrícula do usuário: 1236
Informe o nome do usuário: Maria
Informe o login do usuário: maria
Informe a senha do usuário: maria1234
Informe a matrícula do professor: 20102345
Informe o nome do professor: Jose
Informe o login do professor: jose
Informe a senha do professor: jose3456
Informe a matrícula do aluno: 20123456
Informe o nome do aluno: Marcos
Informe o login do aluno: marcos
Informe a senha do aluno: marc9807

Matrícula: 1236
Nome: Maria
Login: maria

+++++ Dados do Professor +++++

Matrícula: 20102345
Nome: Jose
Login: jose

Matrícula: 20123456
Nome: Marcos
Login: marcos
```

Analisando o resultado do teste acima, verificamos que para o Professor o método `exibeDados()` teve um comportamento diferente. Ele apresentou a mensagem: “Dados do Professor” antes de apresentar os dados. Além disso, se por acaso o atributo: `areaAtuacao` possuísse algum valor, este atributo também seria apresentado na tela, reforçando a ideia de comportamento diferente e específico para o Professor. Caso fosse necessário implementar um comportamento específico também para o aluno, utilizaríamos a mesma ideia apresentada para o Professor.

Para concluir este exemplo, vamos alterar a classe de teste para que peça o valor do atributo `areaAtuacao` do Professor, para verificarmos a execução completa do método: `exibeDados()` da classe Professor.

A classe de teste é apresentada na figura abaixo.

```
public class SegundoExemploPolimorfismo
{
    public static void main(String args[])
    {
        //criar um novo usuário no sistema
        Usuario usuario = new Usuario( Teclado.leInt("Informe a matricula do usuário: "),
                                         Teclado.leString("Informe o nome do usuário: "),
                                         Teclado.leString("Informe o login do usuário: "),
                                         Teclado.leString("Informe a senha do usuário: ")
                                         );

        //criar um novo usuário do tipo Professor no sistema
        Professor professor = new Professor( Teclado.leInt("Informe a matricula do professor: "),
                                             Teclado.leString("Informe o nome do professor: "),
                                             Teclado.leString("Informe o login do professor: "),
                                             Teclado.leString("Informe a senha do professor: ")
                                             );
        professor.setAreaAtuacao("Informática"); // atribuindo valor ao atributo: areaAtuacao por meio do método de configuração

        //criar um novo usuário do tipo Aluno no sistema
        Aluno aluno = new Aluno( Teclado.leInt("Informe a matricula do aluno: "),
                                 Teclado.leString("Informe o nome do aluno: "),
                                 Teclado.leString("Informe o login do aluno: "),
                                 Teclado.leString("Informe a senha do aluno: ")
                                 );

        //apresentar os dados de cada usuário criado
        //Usuario
        usuario.exibeDados();
        //Professor
        professor.exibeDados();
        //Aluno
        aluno.exibeDados();
    }
}
```

Após a alteração da classe de teste, compile e execute o método `main`. O comportamento é apresentado na figura abaixo.

```
BlueJ:
Informe a matrícula do usuário: 1236
Informe o nome do usuário: Maria
Informe o login do usuário: maria
Informe a senha do usuário: marial234
Informe a matrícula do professor: 20102345
Informe o nome do professor: Jose
Informe o login do professor: jose
Informe a senha do professor: jose3456
Informe a matrícula do aluno: 20123456
Informe o nome do aluno: Marcos
Informe o login do aluno: marcos
Informe a senha do aluno: marc9807

Matrícula: 1236
Nome: Maria
Login: maria

++++++ Dados do Professor ++++++

Matrícula: 20102345
Nome: Jose
Login: jose
Área de Atuação: Informática

Matrícula: 20123456
Nome: Marcos
Login: marcos
```

2.5 CLASSE ABSTRATA

As classes são as estruturas utilizadas para a criação dos objetos. Quando criamos um objeto é necessário instanciar uma classe por meio do operador: **new**. Este operador indica que um objeto será criado com a estrutura da classe instanciada (chamada). Existem porém, **algumas classes que não podem ser instanciadas**, ou seja, **não podem criar outros objetos**. Estas classes são as **classes abstract**. Para compreender melhor este conceito, vamos retornar ao exemplo das três classes utilizadas nesta prática: Usuario (superclasse), Professor e Aluno (subclasses).

Vamos imaginar que em um sistema, existem estas três classes. Porém neste sistema existe uma regra, todo usuário do sistema deve ser ou Professor ou Aluno. Não existe um usuário que não seja ou do tipo Professor ou do tipo Aluno. Isso significa que não podemos criar objetos por meio da classe Usuario, portanto, devemos torná-la abstrata. Para representar a classe abstrata no código Java utilizamos a palavra reservada: **abstract** na assinatura da superclasse, conforme a sintaxe abaixo:

```
//a declaração superclasse possui a palavra: abstract
public abstract class Usuario {
    ...
}
```

Quando fizermos esta modificação na nossa classe: Usuário não será mais possível instanciar objetos do tipo Usuarios apenas. Teremos que utilizar ou a classe Professor ou a classe Aluno, que também são Usuarios porém especializados.

2.6 TERCEIRO EXEMPLO UTILIZANDO CLASSE ABSTRATA

Vamos então alterar a nossa classe: Usuario para praticar o uso de **classes abstratas**. Para isso basta acrescentarmos a palavra: **abstract** na assinatura da superclasse, conforme figura abaixo.

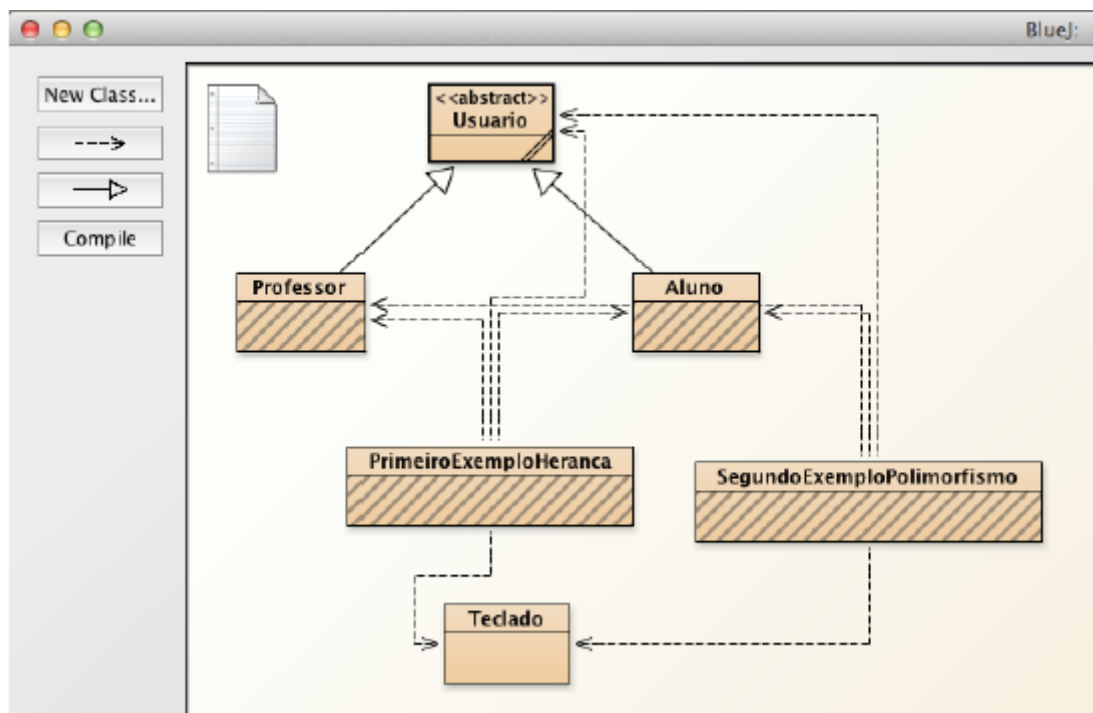
```

public abstract class Usuario
{
    // atributos comuns para todos os usuários
    private int matricula;
    private String nome;
    private String login;
    private String senha;

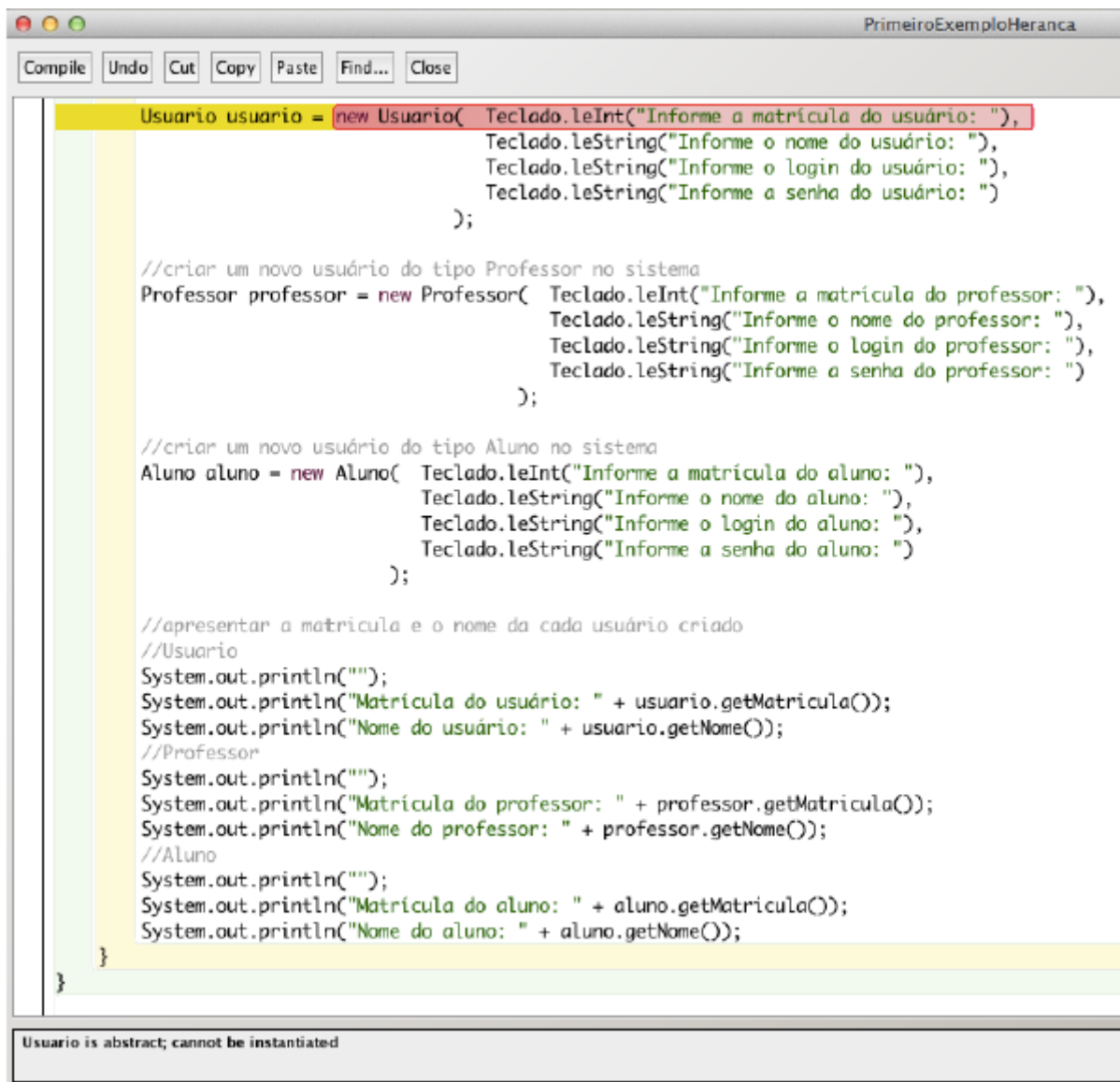
    //método construtor para todos os usuários
    public Usuario(int mat, String nom, String log, String sen)
    {
        matricula = mat;
        nome = nom;
        login = log;
        senha = sen;
    }
}

```

Após a modificação da classe Usuario vamos compilar e fechar o editor da classe. Observe as classes da tela do BlueJ, conforme figura abaixo.



Verifique que após a alteração da classe Usuario, todas as classes dependentes precisam novamente ser compiladas. Vamos fazer isso clicando no botão: [Compile] no lado esquerdo da tela. Ao tentar fazer a compilação das classes, o BlueJ vai apresentar a seguinte tela:



```
Usuario usuario = new Usuario( Teclado.leInt("Informe a matrícula do usuário: "),
                                Teclado.leString("Informe o nome do usuário: "),
                                Teclado.leString("Informe o login do usuário: "),
                                Teclado.leString("Informe a senha do usuário: ")
                                );

//criar um novo usuário do tipo Professor no sistema
Professor professor = new Professor( Teclado.leInt("Informe a matrícula do professor: "),
                                      Teclado.leString("Informe o nome do professor: "),
                                      Teclado.leString("Informe o login do professor: "),
                                      Teclado.leString("Informe a senha do professor: ")
                                      );

//criar um novo usuário do tipo Aluno no sistema
Aluno aluno = new Aluno( Teclado.leInt("Informe a matrícula do aluno: "),
                         Teclado.leString("Informe o nome do aluno: "),
                         Teclado.leString("Informe o login do aluno: "),
                         Teclado.leString("Informe a senha do aluno: ")
                         );

//apresentar a matrícula e o nome da cada usuário criado
//Usuario
System.out.println("");
System.out.println("Matrícula do usuário: " + usuario.getMatricula());
System.out.println("Nome do usuário: " + usuario.getNome());
//Professor
System.out.println("");
System.out.println("Matrícula do professor: " + professor.getMatricula());
System.out.println("Nome do professor: " + professor.getNome());
//Aluno
System.out.println("");
System.out.println("Matrícula do aluno: " + aluno.getMatricula());
System.out.println("Nome do aluno: " + aluno.getNome());
}
```

Usuario is abstract; cannot be instantiated

Mas o que isto significa? Vamos entender o que o BlueJ está nos dizendo. Esta é a nossa primeira classe de teste que criamos, a classe: PrimeiroExemploHeranca. Veja que o BlueJ destacou a linha onde um objeto do tipo Usuario é criado. O BlueJ também apresentou a seguinte mensagem de erro ao final da tela: *"Usuario is abstract; cannot be instantiated"*. Traduzindo o erro, o BlueJ está nos informando que a classe: Usuario é abstrata e, portanto, não podemos criar objetos com esta classe. Interessante não é mesmo? Então, para resolvermos este problema, vamos comentar todas as instruções que são específicas da classe usuário e executar novamente para ver se o comportamento muda.

```
PrimeiroExemploHeranca
Compile Undo Cut Copy Paste Find... Close

public class PrimeiroExemploHeranca
{
    public static void main(String args[])
    {
        /*
         * Este trecho foi comentado pois modificamos a classe Usuario para ser abstrata
         //criar um novo usuário no sistema
        Usuario usuario = new Usuario( Teclado.leInt("Informe a matrícula do usuário: "),
                                        Teclado.leString("Informe o nome do usuário: "),
                                        Teclado.leString("Informe o login do usuário: "),
                                        Teclado.leString("Informe a senha do usuário: ")
                                        );

        */

        //criar um novo usuário do tipo Professor no sistema
        Professor professor = new Professor( Teclado.leInt("Informe a matrícula do professor: "),
                                             Teclado.leString("Informe o nome do professor: "),
                                             Teclado.leString("Informe o login do professor: "),
                                             Teclado.leString("Informe a senha do professor: ")
                                             );

        //criar um novo usuário do tipo Aluno no sistema
        Aluno aluno = new Aluno( Teclado.leInt("Informe a matrícula do aluno: "),
                                Teclado.leString("Informe o nome do aluno: "),
                                Teclado.leString("Informe o login do aluno: "),
                                Teclado.leString("Informe a senha do aluno: ")
                                );

        //apresentar a matricula e o nome da cada usuário criado
        /*
         * Este trecho foi comentado pois modificamos a classe Usuario para ser abstrata
        //Usuario
        System.out.println("");
        System.out.println("Matricula do usuário: " + usuario.getMatricula());
        System.out.println("Nome do usuário: " + usuario.getNome());
        */
    }
}
```

Class compiled - no syntax errors

Faça o mesmo para a segunda classe de teste e tente compilar novamente. Veja que agora é possível compilar as classes sem problemas.

